



---

# ATREDIS PARTNERS

---

## Ymax.app 2026 Solidity Contract Assessment Security Assessment Report



Prepared for Ymax, an Agoric Product  
June 1, 2026 (version 1.3)

**Project Team:**  
Technical Testing: Jordan Whitehead and Shaun Mirani  
Technical Editing: Tom Steele  
Project Management: Emely Tijerina



# Table of Contents

Engagement Overview ..... 3

    Assessment Components and Objectives..... 3

Engagement Tasks ..... 5

Executive Summary ..... 7

    Key Conclusions ..... 7

    Findings Overview ..... 7

Platform Overview ..... 9

Findings and Recommendations ..... 13

    Findings Summary ..... 13

    Findings Detail ..... 13

    Use of abi.decode on Improperly Encoded Data ..... 14

    Lack of Router Parameter Normalization ..... 17

    Factory Accepts Empty Principal ..... 19

Appendix I: Assessment Methodology ..... 21

Appendix II: Engagement Team Biographies ..... 24

Appendix III: About Atredis Partners ..... 30

Appendix IV: Referenced Test Cases ..... 31





# Engagement Overview

## Assessment Components and Objectives

Ymax and Agoric Systems Operating Company (“Agoric”, developer of Ymax) recently engaged Atredis Partners (“Atredis”) to perform a 2026 Solidity Contract Assessment of the Ymax remote account Solidity contracts. Objectives included validation that Ymax services were developed and deployed with security best practices in mind, and to obtain third party validation that any significant vulnerabilities present in Ymax’s Solidity contracts were identified for remediation.

Testing was performed from April 20 through May 1, 2026, by Jordan Whitehead and Shaun Mirani of the Atredis Partners team, with Emely Tijerina providing project management and delivery oversight. For Atredis Partners’ assessment methodology, please see [Appendix I](#) of this document, and for team biographies, please see [Appendix II](#). Specific testing components and testing tasks are included below.

| COMPONENT                                     | ENGAGEMENT TASKS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>YMAX 2026 SOLIDITY CONTRACT ASSESSMENT</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|                                               | <ul style="list-style-type: none"> <li>▪ Verify Ethereum Smart Contract               <ul style="list-style-type: none"> <li>○ Validate business logic of contract state machines</li> <li>○ Ensure only authorized entities can manage remote accounts</li> <li>○ Test for reentrancy issues in the contract</li> <li>○ Ensure proper deserialization and context for proxied calls</li> <li>○ Test for Gas Limit issues or other Gas misuse</li> <li>○ Test for other malicious contract uses, including the “OWASP Smart Contract Top 10”</li> <li>○ Manual runtime analysis and protocol analysis</li> </ul> </li> <li>▪ Verify integration with Axelar GMP and correct use of the SDK</li> <li>▪ Dynamically test Ethereum Smart Contract with local environment               <ul style="list-style-type: none"> <li>○ <code>agoric-to-axelar-local</code> repository provided for local testing</li> <li>○ Local PoC generation and validation of findings</li> </ul> </li> </ul> |
| <b>REPORTING AND ANALYSIS</b>                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Analysis and Deliverables                     | <ul style="list-style-type: none"> <li>▪ Status Reporting and Realtime Communication</li> <li>▪ Comprehensive Engagement Deliverable</li> <li>▪ Engagement Outbrief and Remediation Review</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

The ultimate goal of the assessment was to provide a clear picture of risks, vulnerabilities, and exposures as they relate to accepted security best practices, such as those created by the National



Institute of Standards and Technology (NIST), Open Web Application Security Project (OWASP), or the Center for Internet Security (CIS). Augmenting these, Atredis Partners also draws on its extensive experience in secure development and in testing high-criticality applications and advanced exploitation.



## Engagement Tasks

---

Based on scoping discussions and documentation review, Atredis Partners has identified the following tasks, at a high level, to be applied to the in-scope targets for the engagement.

### **Application Penetration Testing**

For relevant web applications, APIs, and web services, Atredis performed automated and manual application penetration testing of these components, applying generally accepted testing best practices as derived from OWASP and the Web Application Security Consortium (WASC).

Testing was performed from the perspective of an anonymous intruder, identifying scenarios from the perspective of an opportunistic, Internet-based threat actor with no knowledge of the environment, as well as, from the perspective of a user working to laterally move through the environment to bypass security restrictions and user access levels.

Where relevant, Atredis Partners utilized both automated fuzzing and fault injection frameworks as well as purpose-built, task-specific testing tools tailored to the application and platforms under review.

### **Configuration and Architecture Review**

Atredis Partners performed a high-level review of available documentation and configuration data with an eye toward the overall functional design and soundness of the implementation. A key aspect of this component was identifying gaps in the architecture and design regarding aspects of design that reduce overall defensibility, aimed at pointing out fundamental issues in the application architecture that should be addressed early in the development cycle as opposed to later when the platform is closer to a full production state.

While specific vulnerabilities may be identified during the architecture and configuration review, the intent is less on finding individual defects and more on how the design of a given target affects its overall defensibility. Outcomes of the architecture review will help inform testing objectives throughout the rest of the engagement while also helping the client define a long-term platform maturity and security design roadmap.

### **Source Code Analysis**

Atredis reviewed the in-scope application source code, with an eye for security-relevant software defects. To aid in vulnerability discovery, application components will be mapped out and modeled until a thorough understanding of execution flow, code paths, and application design and architecture is obtained. To aid in this process, the assessment team will engage key stakeholders and members of the development team where it is possible to provide structured walkthroughs and interviews, helping the team rapidly gain an understanding of the application's design and development lifecycle.



## **Status Reporting and Realtime Communication**

Atredis Partners scheduled regular status meetings with client representatives during the project and reported findings in realtime as soon as they are confirmed, via secure communication channels.

## **High Priority Issue Reporting**

For Critical and High severity issues discovered during the engagement, Atredis delivered the details to the client once the vulnerability is confirmed. Atredis worked with the client's team to ensure understanding of the issue, the impact, and the proposed mitigation strategy.

## **Status Reports and Meetings**

Atredis delivered, at a minimum, weekly status reports to the client, and (if requested), scheduled status meetings during the engagement. Standard Atredis status reports include project overview and tracking information such as percentage complete on each phase of the testing process. Reporting and meetings deliver relevant issues discovered that week as well as provide remediation guidance and additional information to the client team.

## **Primary Engagement Deliverable**

Standard Atredis deliverables include an executive overview of the project and the results, a detailed analysis of the functionality of the test target, and a complete listing of issues discovered with mitigation strategies and additional contextual information.

The primary deliverable will document the entire understanding of the test target as understood by Atredis, including the functionality of subsystems that are integral to the issues discovered in the findings section. Typical deliverables are 40-120 pages of documentation surrounding the project and include both executive summaries and technical details of the target, the issues discovered, mitigation suggestions, and other content relevant to the target and client. Atredis can provide a standard sample deliverable upon request.

Atredis can add to or modify the format of Atredis' standard engagement deliverable to meet the needs of the client upon reasonable request.



## Executive Summary

---

The Ymax remote account contracts provide a mechanism to support cross-chain calls that result in the creation and use of `RemoteAccount` proxy accounts on multiple Ethereum Virtual Machine (EVM) destination chains. The smart contracts in scope for testing were not lengthy or convoluted but contained security-critical logic. Errors within smart contracts have historically been a large source of financial risk.

Testing was performed with careful automated and manual review, as well as the addition of tests for dynamic verification. The scope for this testing contained only the Solidity contracts that act as part of the remote account system. Notably, the relevant Cosmos contracts, the Axelar GMP system, and the Permit2 contracts used were not in scope for review. Where notable, this document points out critical areas where the remote account contracts depend on the correctness of other contracts.

The Ymax Solidity contracts have been developed to add additional control and growth mechanisms, giving privileged parties the ability to add additional message routers, and potentially decouple the system from sole reliance on Axelar GMP and other dependencies. Verification of the routing and management logic was the key focus for the testers.

## Key Conclusions

Overall, Atredis Partners found Ymax's Solidity contracts to be well-architected from a security perspective, with care taken to avoid scenarios that could lead to loss of funds or account takeover. The concise contracts did not add bloat that would increase attack surface and Ethereum gas prices when invoking the contracts.

Atredis Partners performed extensive manual and automated testing of Ymax's contracts, finding no exploitable security issues. Runtime verification was done by adding custom test logic to inform and confirm static understanding.

As in any security assessment, some general areas for improvement were noted, but overall Atredis Partners would rate Ymax's contracts as sound from a security perspective and well-aligned with modern secure blockchain development practices.

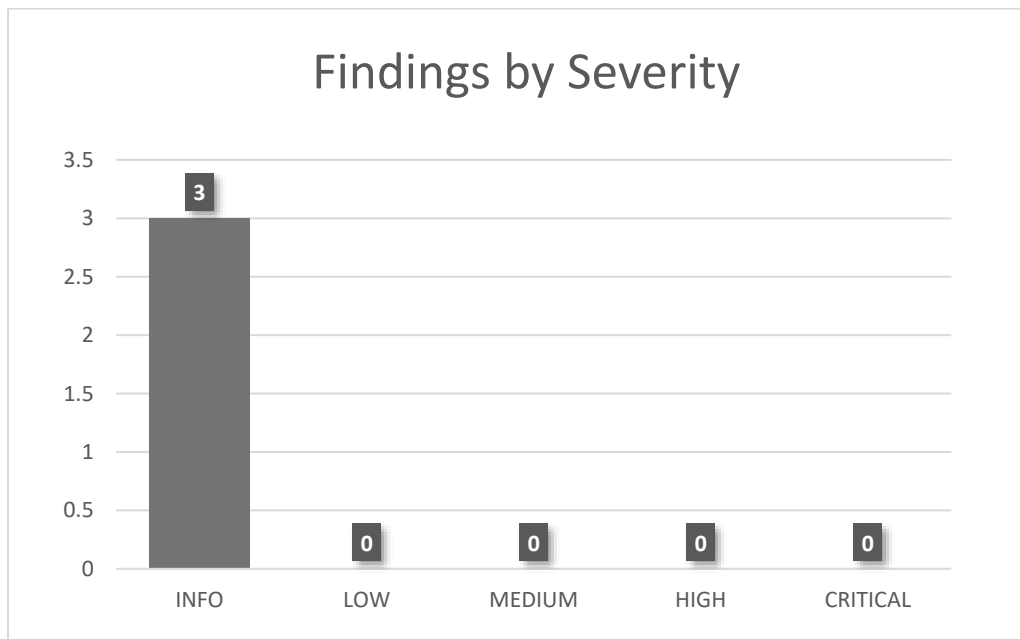
## Findings Overview

In performing testing for this assessment, Atredis Partners identified **zero (0) critical**, **zero (0) high**, **zero (0) medium**, **zero (0) low** severity findings, and **three (3) informational** findings. No high or critical severity findings were noted. As stated earlier, none of these issues constitute a potential for direct compromise.

Atredis defines vulnerability severity ranking as follows:



- **Critical:** These vulnerabilities expose systems and applications to immediate threat of compromise by a dedicated or opportunistic attacker.
- **High:** These vulnerabilities entail greater effort to exploit compared to Critical severity issues or otherwise have relevant limitations preventing widespread or immediate exploitation. They may result in successful compromise within a relatively short time.
- **Medium:** These vulnerabilities may not directly lead to compromise but provide useful information or functionality to an attacker. Medium severity vulnerabilities may be chained together with other issues to ultimately result in greater impact to the target.
- **Low:** These vulnerabilities fall below the impact threshold of Mediums and are typically related to improper disclosure of information or minor increases in attack surface. They may provide attackers with non-public information that can lead to additional attack vectors or lower the level of effort necessary to exploit a different vulnerability.
- **Info:** These findings consist of improper security practices or other issues that do not currently present security risks but could expose systems or applications to vulnerabilities in the future if not addressed.







## Platform Overview

---

Ymax's remote account system depends on three main contracts: `RemoteAccountAxelarRouter`, `RemoteAccountFactory`, and finally `RemoteAccount`. These contracts are deployed so that the trusted account on the Agoric chain (the Portfolio Manager) is registered as the "Factory Principal Account".

### RemoteAccountAxelarRouter

The `RemoteAccountAxelarRouter.sol` contract is the entry point for the system. The contract inherits from `AxelarExecutable`, integrating with the Axelar GMP (General Message Passing) contracts to allow messages to be sent from the Agoric chain to the contract's chain. While the contract has a number of functions marked external, these functions all check that the caller is the router itself, preventing unexpected interactions with the router outside of the Axelar messages.

The remote account system depends on the correctness of Axelar. If messages could be spoofed with misleading `sourceAddress` parameters or replayed messages, it would undermine the security of the Ymax remote accounts. The router contract assumes it can trust the Axelar contracts and does not (or cannot) verify those sensitive parameters.

Specifically, the `RemoteAccountAxelarRouter` contract does not check whether a transaction's payload-level identifier (the `txId` field) has already been seen before calling `processInstruction` and returning the result. See the `RemoteAccountDuplicateSend.spec.ts` test included in [Appendix IV](#) for more information on the lack of ordering and uniqueness checks within the router contract itself.

If the system should not depend on Axelar's provided uniqueness verification checks, consider modifying `RemoteAccountAxelarRouter` and all future router contracts to track seen `txIds` immediately after decoding the payload. Upon encountering an expired `txId`, have the contract emit an event indicating a duplicate was dropped and return without executing `processInstruction`. Additionally, ensure that `txId` is unique per GMP operation on the Agoric sender side.

Likewise, the remote account system depends on the Portfolio Manager on the Agoric chain to have correct logic. Mistakes in that contract could lead to unexpected side effects within the remote account contracts. The portfolio manager must ensure to not double send sensitive messages, especially if future development allows multiple router contracts to be active at the same time. The logic around the creation of Permit2 payment methods must also ensure the sender and recipients are correctly specified.

The instructions provided by the `RemoteAccountAxelarRouter` contract allow non-Portfolio Manager contracts to create remote accounts via the `RemoteAccountFactory` contract and execute batched



instructions via their corresponding `RemoteAccounts`. The Portfolio Manager can perform additional actions through `RemoteAccountFactory`, as described below.

```
function checkInstructionSelector(bytes4 selector) internal pure {
    if (
        selector != RemoteAccountAxelarRouter.processRemoteAccountExecuteInstruction.selector &&
        selector != RemoteAccountAxelarRouter.processProvideRemoteAccountInstruction.selector &&
        selector != RemoteAccountAxelarRouter.processAuthorizeRouterInstruction.selector &&
        selector != RemoteAccountAxelarRouter.processDeauthorizeRouterInstruction.selector &&
        selector != RemoteAccountAxelarRouter.processConfirmVettingAuthorityInstruction.selector
    ) {
        revert InvalidInstructionSelector(selector);
    }
}
```

### Available “instructions” Provided by RemoteAccountAxelarRouter

One informational issue was noted in the gas calculation logic used by the `RemoteAccountAxelarRouter` contract. See [Use of abi.decode on Improperly Encoded Data](#) for details.

The architecture of the remote account system allows for the addition of other authorized routers, though the current implementation only supports the `RemoteAccountAxelarRouter`. Future development should ensure that additional routers provide the exact same normalization of parameters and ordering guarantees as provided by Axelar – see the informational issue [Lack of Router Parameter Normalization](#) for more details.

## RemoteAccountFactory

The `RemoteAccountFactory` contract is a contract that handles the logic of `RemoteAccount` creation, router activation, and other maintenance tasks. It correctly checks that any exposed mutating endpoint is limited to either the vetting authority or an authorized router or cannot have malicious side-effects.

For example, the `provideRemoteAccount` function can be called by any external contract, but the resulting `RemoteAccount` can only be interacted with by the owner of the account as reported by the router.

```
/**
```



```
* @notice Provide a RemoteAccount - creates if new, verifies if exists
* @dev Idempotent: calling multiple times with same params is safe.
*     Since accounts immutably delegate authorization through this factory,
*     there is no account state to verify (we rely on deterministic address
*     derivation of the remote accounts, and atomic initialization).
*     This method is intentionally public as the created remote account is
*     intrinsically tied to its principal and this factory. No other party
*     than the designated principal can operate the account, and only
*     through an authorized router.
* @param principalAccount The principal account string for the RemoteAccount
* @param expectedAddress The expected CREATE2 address (for verification)
* @return created true if the RemoteAccount was created, false if it was pre-existing
*/
function provideRemoteAccount(
    string calldata principalAccount,
    address expectedAddress
) external override returns (bool) {
    bytes32 salt = _verifyRemoteAccountAddress(principalAccount, expectedAddress);

    if (expectedAddress.code.length != 0) {
        return false;
    }

    _createRemoteAccount(principalAccount, salt, expectedAddress);
    return true;
}
```

### **provideRemoteAccount in RemoteAccountFactory.sol**

The nature of the CREATE2 mechanism used by the `RemoteAccountFactory` prevents these accounts from being front-run by a malicious actor seeking to place a malicious contract in place of an account that should be newly created. CREATE2 requires a deterministic address be chosen based on a hash that includes the calling contract's address and the bytes of the created contract. This means it is cryptographically infeasible to find another input that will result in the same deterministic address.

The Factory contains the logic used to authorize or un-vet routers, as well as manage the vetting authority. All EVM router contracts in the Authorized state are currently equally privileged. Any authorized router is able to manage all `RemoteAccount` instances and perform all administrative



actions, including authorizing/deauthorizing other routers. A compromise of a single router (via a vulnerability in the contract code or the underlying relayer's authentication scheme) therefore affects all managed accounts and the integrity of the routing system itself.

With a small set of trusted routers that use a single relay network (Axelar), this design carries little risk on its own. However, if Ymax were to significantly scale the pool of routers, with some using different relay networks with differing levels of security, the system's overall security would be bounded by the most vulnerable authorized router.

If significant scaling to additional routers and routing layers is planned, Ymax should create separate account operation and administrative roles and limit the set of routers authorized to take administrative actions.

## RemoteAccount

The `RemoteAccount` contract is an EIP-1167 implementation contract. Its own initializers are locked, so that the contract can act only as the implementation for proxy contracts created by the factory. These proxied contracts do not contain much controllable state variables but can receive and hold funds. The core of its logic is in the `executeCalls` function, which securely ensures that it is only called from an authorized router. This call allows remote users to perform actions on the destination chain from the Agoric chain. Further consistency is required due to the proxy's address being based on the trusted cross-chain `sourceAddress` parameter. Without being able to spoof a source address, an attacker should not be able to perform actions as any account other than their own. This results in a remote account that should only be usable by the associated principal on the Agoric chain.



## Findings and Recommendations

The following section outlines findings identified via manual and automated testing over the course of this engagement. Where necessary, specific artifacts to validate or replicate issues are included, as well as Atredis Partners' views on finding severity and recommended remediation.

### Findings Summary

The tables below summarize the number and severity of the unique issues identified throughout the engagement.

| CRITICAL | HIGH | MEDIUM | LOW | INFO |
|----------|------|--------|-----|------|
| 0        | 0    | 0      | 0   | 3    |

### Findings Detail

| FINDING NAME                                 | SEVERITY |
|----------------------------------------------|----------|
| Use of abi.decode on Improperly Encoded Data | INFO     |
| Lack of Router Parameter Normalization       | INFO     |
| Factory Accepts Empty Principal              | INFO     |



## Use of abi.decode on Improperly Encoded Data

**SEVERITY: INFO**

### Finding Overview

The `RemoteAccountAxelarRouter.processInstruction` method's out-of-gas logic uses `abi.decode` to unpack incorrectly encoded ABI data, leading to a decoding failure and unreachable `SubcallOutOfGas` error reporting.

### Finding Detail

When subcall execution fails, the `RemoteAccountAxelarRouter.processInstruction` contract method implements a basic heuristic to assess whether the failure was caused by gas exhaustion.

If the length of the subcall result is greater than zero, the code attempts to extract the `reason` field. It uses `abi.encodePacked` to prepend 28 zeros to the result data so that the 4-byte error selector field occupies 32 bytes, then unpacks this padded result into the `reason` variable by calling `abi.decode`.

If `reason`'s length is zero, `processInstruction` assumes an out-of-gas error occurred and propagates a `SubcallOutOfGas` error to the caller.

```
if (!success && gasAfter <= (gasBefore / 7)) {
    if (result.length == 0) {
        // The call likely ran out of gas without RemoteAccount interception
        revert SubcallOutOfGas();
    }

    if (bytes4(result) == IRemoteAccount.ContractCallFailed.selector) {
        // Prepend 28 bytes of zeros to 'complete' the custom error selector into a 32-byte
        word.
        // This allows abi.decode to treat the selector as the first argument.
        bytes memory paddedData = abi.encodePacked(new bytes(28), result);

        // Now we decode including the error selector as the first argument.
        (, , , bytes memory reason) = abi.decode(
            paddedData,
            (bytes4, address, bytes4, uint224, bytes)
        );

        if (reason.length == 0) {
            // The call made by RemoteAccount likely ran out of gas
            revert SubcallOutOfGas();
        }
    }
}
```



```
    }  
  }  
}
```

### processInstruction Out-of-Gas Handling in RemoteAccountAxelarRouter.sol

However, left-padding the result data shifts the position of the `reason` field to the right without adjusting its offset in the ABI-encoded data accordingly. As a result, the `abi.decode` call throws an exception on the improperly encoded data, leading to an empty revert. The `revert SubcallOutOfGas()` statement is therefore unreachable, and callers have no insight into the cause of the error.

The following test demonstrates the issue, tuning gas consumption to hit the `processInstruction` branch shown above. It expects a `SubcallOutOfGas` revert, but the decoding bug causes an empty revert instead. The test fails as a result.

```
// ...  
it('Branch 2 should revert with SubcallOutOfGas', async () => {  
  const mc = contractWithCallMetadata(  
    makeEvmContract(multicallAbi),  
    multicallTarget.target.toString() as 0x${string},  
  );  
  
  // Tune gas to hit target branch  
  const multiCalls: ContractCall[] = [  
    mc.burnGas(50000n),  
    mc.setValue.with({ gasLimit: 50n })(42n),  
  ];  
  
  const receipt = await route(testLCA, {  
    async doExecute(commandId, sourceChain, sourceAddress, payload) {  
      const gasEstimate = await this.execute.estimateGas(  
        commandId, sourceChain, sourceAddress, payload,  
      );  
      return this.execute(commandId, sourceChain, sourceAddress, payload, {  
        gasLimit: (gasEstimate * 70n) / 100n,  
      });  
    },  
  }).doRemoteAccountExecute({ multiCalls });
```



```
// Should revert with SubcallOutOfGas, but the decoding bug causes an empty revert and the assertion to fail
await expect(receipt).to.be.revertedWithCustomError(router, 'SubcallOutOfGas');
});
```

### SubcallOutOfGasUnreachable.spec.ts Expects SubcallOutOfGas Revert

Test execution and output:

```
$ cd agoric-to-axelar-local/packages/axelar-local-dev-cosmos && hardhat test
src/__tests__/SubcallOutOfGasUnreachable.spec.ts

> L-4: OOG Heuristic Branch 2 – abi.decode revert in actual router
  1) Branch 2 should revert with SubcallOutOfGas

0 passing (2s)
1 failing

1) L-4: OOG Heuristic Branch 2 – abi.decode revert in actual router
   Branch 2 should revert with SubcallOutOfGas:
     AssertionError: Expected transaction to be reverted with custom error 'SubcallOutOfGas',
but it reverted without a reason
```

### Test Failure

## Recommendation(s)

Replace `abi.decode` with an `mload` instruction that reads the `reason` field at its fixed offset within `result`.

Write tests to validate that the out-of-gas handling code works as expected.

## References

SCWE-011: Insecure ABI Encoding and Decoding:

<https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-011/>





## Lack of Router Parameter Normalization

**SEVERITY: INFO**

### Finding Overview

The Remote Account contracts depend on a `sourceAddress` parameter passed to the `RemoteAccountAxelarRouter` contract via the Axelar GMP system. Addition of future routers may normalize this address differently, which could lead to unexpected loss of funds or possible aliasing issues.

### Finding Detail

The `RemoteAccountAxelarRouter` contract accepts cross chain messages sent through the Axelar GMP (General Message Passing) system. The contract implements the `_execute` entry point that is called by the base contract, with a `sourceAddress` and `sourceChain` parameter.

```
function _execute(  
    bytes32 /*commandId*/,  
    string calldata sourceChain,  
    string calldata sourceAddress,  
    bytes calldata payload  
) internal override {  
    //...
```

**From RemoteAccountAxelarRouter.sol**

The `sourceAddress` parameter is sensitive; it is what the `RemoteAccount` system uses to verify the cross-chain principal and provide correct `RemoteAccount` proxy addresses.

```
// Provide or verify the remote account exists at the expected address  
factory.provideRemoteAccount(sourceAddress, expectedAccountAddress);
```

**From processRemoteAccountExecuteInstruction in RemoteAccountAxelarRouter.sol**

If future routers are added to the system that do not normalize the reported `sourceAddress` in the same way as Axelar, then it may lead to accounts no longer being usable through the new router. In unlikely scenarios, failure to properly normalize could even result in aliasing of reported `sourceAddress` and remote account takeover.

### Recommendation(s)

Future development should ensure that additional routers are guaranteed to normalize addresses in the same way as Axelar, or normalization checks should be added to the account factory logic.



## References

CWE-20: Improper Input Validation:

<https://cwe.mitre.org/data/definitions/20.html>



## Factory Accepts Empty Principal

**SEVERITY: INFO**

### Finding Overview

The `RemoteAccountFactory` contract creates remote accounts that can only be fully used by an account with the specified `sourceAddress` passed to the router. If a router could ever provide an empty address, the Factory will still allow its use. Defense-in-depth should require checks against this edge-case, and verification of the `sourceAddress` length.

### Finding Detail

When the authorized router accepts a cross-chain message, it reports the `sourceAddress` of the caller. This address is used by the remote account system to ensure that the caller is the owner of the remote account. The logic in `RemoteAccountFactory` uses this account string to create a hash and a salt, which are used for deterministically choosing the address of the `RemoteAccount` contract to be created or used.

```
function _getSalt(string calldata principalAccount) internal pure returns (bytes32) {
    return keccak256(bytes(principalAccount));
}

/**
 * @notice Verify a principal account matches the factory's designated principal
 * @dev Only checks the account part not the caip2. The intent is to help owners
 *      disambiguate between the factory and a RemoteAccount.
 *      Reverts if the principal does not match.
 * @param expectedPrincipalAccount The expected address of the factory's principal
 */
function verifyFactoryPrincipalAccount(
    string calldata expectedPrincipalAccount
) external view override {
    if (_getSalt(expectedPrincipalAccount) != _principalSalt) {
        revert PrincipalAccountMismatch(expectedPrincipalAccount,
factoryPrincipalAccount);
    }
}
```

#### A Portion of RemoteAccountFactory.sol

If logical mistakes in dependencies such as routers or cross chain message services allow for malformed or empty principal accounts to be used, then an attacker could possibly craft a situation



which would cause a user to mistakenly use the account associated with an empty or malformed `sourceAddress`. The attacker could then retrieve misplaced funds, or perform other malicious actions.

The security of the `RemoteAccountFactory` logic hinges on the correctness of the string specifying the principal parameter. While this issue is not known to be exploitable; additional logic may be warranted to mitigate against misuse of this sensitive parameter.

## Recommendation(s)

While no technique is currently known to allow the Axelar GMP messages `sourceAddress` to be reported as having a zero-size string, refusing to handle malformed addresses can mitigate against many unknown issues. The router or the factory should have logic checking that the `sourceAddress` (sometimes used as `principalAccount` string in the factory code) strictly matches the expected string length.

## References

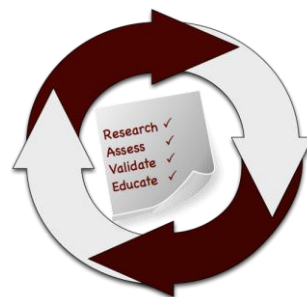
CWE-20: Improper Input Validation:

<https://cwe.mitre.org/data/definitions/20.html>



## Appendix I: Assessment Methodology

Atredis Partners draws on our extensive experience in penetration testing, reverse engineering, hardware/software exploitation, and embedded systems design to tailor each assessment to the specific targets, attacker profile, and threat scenarios relevant to our client's business drivers and agreed upon rules of engagement.



Where applicable, we also draw on and reference specific industry best practices, regulations, and principles of sound systems and software design to help our clients improve their products while simultaneously making them more stable and secure.

Our team takes guidance from industry-wide standards and practices such as the National Institute of Standards and Technology's (NIST) Special Publications, the Open Web Application Security Project (OWASP), and the Center for Internet Security (CIS).

Throughout the engagement, we communicate findings as they are identified and validated, and schedule ongoing engagement meetings and touchpoints, keeping our process open and transparent and working closely with our clients to focus testing efforts where they provide the most value.

In most engagements, our primary focus is on creating purpose-built test suites and toolchains to evaluate the target, but we do utilize off-the-shelf tools where applicable as well, both for general patch audit and best practice validation as well as to ensure a comprehensive and consistent baseline is obtained. Additionally, Atredis consultants may utilize authorized Generative Artificial Intelligence (GenAI) and/or Large Language Models (LLMs) to enhance workflows and client services in accordance with Atredis GenAI/LLM acceptable use policies.

### Research and Profiling Phase

Our research-driven approach to testing begins with a detailed examination of the target, where we model the behavior of the application, network, and software components in their default state. We map out hosts and network services, patch levels, and application versions. We frequently use a number of private and public data sources to collect Open-Source Intelligence about the target and collaborate with client personnel to further inform our testing objectives.

For network and web application assessments, we perform network and host discovery as well as map out all available application interfaces and inputs. For hardware assessments, we study design and implementation, down to a circuit-debugging level. In reviewing source code or compiled application code, we map out application flow and call trees and develop a solid working



understanding of how the application behaves, thus helping focus our validation and testing efforts on areas where vulnerabilities might have the highest impact to the application's security or integrity.

## **Analysis and Instrumentation Phase**

Once we have developed a thorough understanding of the target, we use a number of specialized and custom-developed tools to perform vulnerability discovery as well as binary, protocol, and runtime analysis, frequently creating engagement-specific software tools which we share with our clients at the close of any engagement.

We identify and implement means to monitor and instrument the behavior of the target, utilizing debugging, decompilation and runtime analysis, as well as making use of memory and filesystem forensics analysis to create a comprehensive attack modeling testbed. Where they exist, we also use common off-the-shelf, open-source and any extant vendor-proprietary tools to aid in testing and evaluation.

## **Validation and Attack Phase**

Using our understanding of the target, our team creates a series of highly specific attack and fault injection test cases and scenarios. Our selection of test cases and testing viewpoints are based on our understanding of which approaches are most relevant to the target and will gain results in the most efficient manner and built in collaboration with our client during the engagement.

Once our test cases are validated and specific attacks are confirmed, we create proof-of-concept artifacts and pursue confirmed attacks to identify extent of potential damage, risk to the environment, and reliability of each attack scenario. We also gather all the necessary data to confirm vulnerabilities identified and work to identify and document specific root causes and all relevant instances in software, hardware, or firmware where a given issue exists.

## **Education and Evidentiary Phase**

At the conclusion of active testing, our team gathers all raw data, relevant custom tool chains, and applicable testing artifacts, parses and normalizes these results, and presents an initial finding brief to our clients, so that remediation can begin while a more formal document is created. Additionally, our team shares confirmed high-risk findings throughout the engagement so that our clients may begin to address any critical issues as soon as they are identified.

After our brief and initial findings review, we develop a detailed research deliverable report that provides not only our findings and recommendations but also an open and transparent narrative about our testing process, observations and specific challenges in developing attacks against our targets, from the real-world perspective of a skilled, motivated attacker.



## Automation and Off-The-Shelf Tools

Where applicable or useful, our team does utilize licensed and open-source software to aid us throughout the evaluation process. These tools and their output are considered secondary to manual human analysis, but nonetheless provide a valuable secondary source of data, after careful validation and reduction of false positives.

For runtime analysis and debugging, we rely extensively on Hopper, IDA Pro and Hex-Rays, as well as platform-specific runtime debuggers, and develop fuzzing, memory analysis, and other testing tools primarily in Ruby and Python.

In source auditing, we typically work in Visual Studio, Xcode and Eclipse IDE, as well as other markup tools. For automated source code analysis, we will typically use the most appropriate tool chain for the target, unless client preference dictates another tool.

Network discovery and exploitation make use of Nessus, Metasploit, and other open source scanning tools, again deferring to client preference where applicable. Web application runtime analysis relies extensively on the Burp Suite, Fuzzer and Scanner, as well as purpose-built automation tools built in Go, Ruby and Python.

## Engagement Deliverables

Atredis Partners deliverables include a detailed overview of testing steps and testing dates, as well as our understanding of the specific risk profile developed by performing the objectives of the given engagement.

In the engagement summary we focus on “big picture” recommendations and a high-level overview of shared attributes of vulnerabilities identified and organizational-level recommendations that might address these findings.

In the findings section of the document, we provide detailed information about vulnerabilities identified, provide relevant steps and proof-of-concept code to replicate these findings, and our recommended approach to remediate the issues, developing these recommendations collaboratively with our clients before finalization of the document.

Our team typically makes use of both DREAD and NIST CVE for risk scoring and naming, but as part of our charter as a client-driven and collaborative consultancy, we can vary our scoring model to a given client’s preferred risk model, and in many cases will create our findings using the client’s internal findings templates, if requested.

Sample deliverables can be provided upon request, but due to the highly specific and confidential nature of Atredis Partners’ work, these deliverables will be heavily sanitized and give only a very general sense of the document structure.



## Appendix II: Engagement Team Biographies

---

### **Shawn Moyer, Founding Partner and CEO**

Shawn Moyer scopes, plans, and coordinates security research and consulting projects for the Atredis Partners team, including reverse engineering, binary analysis, advanced penetration testing, and private vulnerability research. As CEO, Shawn works with the Atredis leadership team to build and grow the Atredis culture, making Atredis Partners a home for some of the best minds in information security, and ensuring Atredis continues to deliver research and consulting services that exceed our client's expectations.

### **Experience**

Shawn brings over 25 years of experience in information security, with an extensive background in penetration testing, advanced security research including extensive work in mobile and Smart Grid security, as well as advanced threat modeling and embedded reverse engineering.

Shawn has served as a team lead and consultant in enterprise security for numerous large initiatives in the financial sector and the federal government, including IBM Internet Security Systems' X-Force, MasterCard, a large Federal agency, and Wells Fargo Securities, all focusing on emerging network and application attacks and defenses.

In 2010, Shawn created Accuvant Labs' Applied Research practice, delivering advanced research-driven consulting to numerous clients on mobile platforms, critical infrastructure, medical devices and countless other targets, growing the practice 1800% in its first year.

Prior to Accuvant, Shawn helped develop FishNet Security's penetration testing team as a principal security consultant, growing red team offerings and advanced penetration testing services, while being twice selected as a consulting MVP.

### **Key Accomplishments**

Shawn has written on emerging threats and other topics for Information Security Magazine and ZDNet, and his research has been featured in the Washington Post, BusinessWeek, NPR and the New York Times. Shawn is a twelve-time speaker at the Black Hat Briefings and has been an invited speaker at other notable security conferences around the world.

Shawn is likely best known for delivering the first public research on social network security, pointing out much of the threat landscape still exists on social network platforms today. Shawn also co-authored an analysis of the state of the art in web browser exploit mitigation, creating the first in-depth comparison of browser security models along with Dr. Charlie Miller, Chris Valasek, Ryan Smith, Joshua Drake, and Paul Mehta.





Shawn studied Computer and Network Information Systems at Missouri University and the University of Louisiana at Lafayette, holds numerous information security certifications, and has been a frequent presenter at national and international security industry conferences.



## **Jordan Whitehead, Research Consulting Director**

Jordan Whitehead specializes in vulnerability research and binary exploitation. Jordan is able to quickly dive into large systems and find key weaknesses as a result of his significant experience in operating system internals.

### **Experience**

During Jordan's Computer Engineering degree schooling, he created and instructed collegiate courses and clubs on computer security. After college he worked as a CNO developer for ManTech International, developing tools and capabilities that involved deep exploration into modern operating systems for exploitable weaknesses. While in that position, Jordan also continued to help create and instruct a number of formal reverse engineering and exploitation courses. These courses detailed the system internals for Windows, Linux, and Android. He has worked with research teams developing custom virtualization and emulation tooling that enabled researchers to better assess otherwise unreachable systems.

### **Key Accomplishments**

Jordan has helped publish papers at top academic conferences on computer security, including Usenix Security Symposium. He has also developed open-source tools related to vulnerability research and secure software. These include peer-reviewed tools that have helped provide useable security and trust on Linux and Windows platforms.



## **Shaun Mirani, Senior Research Consultant**

Shaun is an experienced application security professional with interests in cryptography, static analysis, fuzzing, and exploit development. His passion for software engineering and computer science drives him to search for the fundamental causes of security vulnerabilities and to design the most effective remediations.

### **Experience**

Shaun brings nearly a decade of industry experience to Atredis Partners, having previously worked as a Security Engineer at Trail of Bits and a Senior Security Analyst at Independent Security Evaluators. Over the course of his career, he has performed a wide variety of application security assessments, threat modeling engagements, system design reviews, and black-box penetration tests. He holds a B.S. in Computer Science from the University of Maryland, College Park.

### **Key Accomplishments**

Shaun has led and executed security assessments for clients ranging from Fortune 500 companies to open-source software nonprofit groups, co-presented security research at RSA Conference, and discovered numerous critical vulnerabilities both independently and on behalf of his employers.



## **Tom Steele, Vice President of Research Consulting**

Tom Steele leads and executes application security assessments and adversarial engagements, ranging from source code review to advanced red team assessments.

### **Experience**

Tom has over eight years of professional experience in information security. During that time, his focus has been on executing and innovating both network and application level assessments; with a focus on developing new techniques, tools, and processes that improve collaborative testing, coverage, deterrent bypass, and data exfiltration.

In addition to performing assessments, Tom is also a seasoned software developer, and has an expert knowledge of multiple languages and platforms including Go and Node.js. Tom understands how applications fit together and has used his development experience to develop and maintain many widely used open-source and proprietary tools including Lair, a real-time testing collaboration application, and BurpBuddy, an API for BurpSuite Pro.

Prior to joining Atredis, Tom was a practice manager on Optiv's Attack and Penetration team, where he led a team of consultants, developed and enhanced methodologies, toolsets, and processes, and conducted hundreds of security assessments.

### **Key Accomplishments**

Tom is a contributor to the Node Security Project, where he has assisted with the identification and remediation of many vulnerabilities; both in Node core and in widely deployed libraries. He has consulted leaders working at Fortune 500 companies on how to increase the security of their application frameworks. He has presented and lead training at several conferences including Black Hat, DEF CON, BSides, and DerbyCon and is the Co-Author of No Starch Press' "Black Hat Go".



## **Emely Tijerina, Client Operations Associate**

Emely joined the Operations team in July, 2024. She supports every phase of the project lifecycle at Atredis Partners.

### **Experience**

Prior to joining Atredis Partners, Emely worked in the Federal Government contracting field for 9 years in various capacities. During her time, she spent the last 5 years gaining experience in project management for numerous federal agencies around the U.S. Her experience includes coordinating with cross-functional teams to resolve project issues and mitigate risks and providing detailed project status updates to stakeholders and executive management.

### **Key Accomplishments**

Emely earned a bachelor's degree in Art with an emphasis in Drawing and Illustration from Olivet Nazarene University.



## Appendix III: About Atredis Partners



Founded in 2013, Atredis Partners is made up of some of the greatest minds in Information Security. We've built our brand on a longstanding reputation for performing deeper, more advanced assessments with consistent quality of delivery and execution.

In 2024, Atredis became a 100% [employee-owned](#) company, continuing our mission of building long-standing, sustainable relationships with both our clients and our team members (who have an average tenure of 5 years, unheard of in our industry).

Atredis Partners team members have presented research over fifty times at the BlackHat Briefings conference in Europe, Japan, and the United States over the years, as well as many other notable security conferences. Notable Atredians have included authors of Metasploit, runZero, Kismet, Veil, Lair, and many other Open Source security tools.

Most of our team hold one or more advanced degrees in Computer Science or engineering, as well as many other industry certifications and designations. Atredis team members have authored several books, including *The Android Hacker's Handbook*, *The iOS Hacker's Handbook*, *Wicked Cool Shell Scripts*, *Gray Hat C#*, and *Black Hat Go*.

We've delivered notable public security research for Google, Microsoft, Salesforce, The Linux Foundation, DARPA, the Cloud Native Computing Foundation, the Linux Foundation, and many others, and were the first firm to be named in Qualcomm's Product Security Hall of Fame.

Atredis also delivers advanced risk management consulting, partnering with our research consultants to deliver hybrid engagements that encompass the technical trenches as well as the C-Suite, building mature security programs and helping our clients move beyond checkbox-centric thinking to make informed decisions about real world business and technical risk.

Atredis Partners worked closely with the Open Compute Project to contribute to the [OCP SAFE](#) standard, which takes a risk-based, threat-centric approach to hybrid embedded hardware and firmware assessments, and were among the first three firms to be named as an OCP Safe Security Review Provider.





## Appendix IV: Referenced Test Cases

```
import AxelarGasService from '@axelar-network/axelar-cgp-solidity/artifacts/contracts/gas-
service/AxelarGasService.sol/AxelarGasService.json';
import { expect } from 'chai';
import { Contract } from 'ethers';
import { ethers } from 'hardhat';
import '@nomicfoundation/hardhat-chai-matchers';
import { HardhatEthersSigner } from '@nomicfoundation/hardhat-ethers/signers';
import { gmpRouterContract, padTxId, contractWithCallMetadata } from '../utils/router';
import { makeEvmContract } from '../utils/evm-facade';
import { routed, deployRemoteAccountFactory } from '../lib/utils';
import type { ContractCall } from '../interfaces/router';
import { multicallAbi } from '../interfaces/multicall';

describe('RemoteAccountAxelarRouter - Duplicate Send PoC', () => {
  let owner: HardhatEthersSigner;
  let axelarGateway: Contract;
  let factory: Contract, router: Contract, permit2Mock: Contract;
  let multicallTarget: Contract;

  const abiCoder = new ethers.AbiCoder();
  const sourceChain = 'agoric';
  const portfolioContractCaip2 = 'cosmos:agoric-3';
  const portfolioContractAccount = 'agoric1routerlca123456789abcdefghijklmnpqrs';
  const lca = 'agoric1duplicatepoc1234567890abcdefghijkl';

  let route: ReturnType<typeof routed>;

  before(async () => {
    [owner] = await ethers.getSigners();

    const GasServiceFactory = await ethers.getContractFactory(
      AxelarGasService.abi,
      AxelarGasService.bytecode,
    );
    await GasServiceFactory.deploy(owner.address);
```



```
const TokenDeployerFactory = await ethers.getContractFactory('TokenDeployer');
const tokenDeployer = await TokenDeployerFactory.deploy();

const AuthFactory = await ethers.getContractFactory('AxelarAuthWeighted');
const auth = await AuthFactory.deploy([
  abiCoder.encode(['address[]', 'uint256[]', 'uint256'], [[owner.address], [1],
1]),
]);

const GatewayFactory = await ethers.getContractFactory('AxelarGateway');
axelarGateway = await GatewayFactory.deploy(auth.target, tokenDeployer.target);

permit2Mock = await (await ethers.getContractFactory('MockPermit2')).deploy();

factory = await deployRemoteAccountFactory(
  portfolioContractCaip2,
  portfolioContractAccount,
  owner.address,
);

const RouterContract = await ethers.getContractFactory('RemoteAccountAxelarRouter');
router = await RouterContract.deploy(
  axelarGateway.target,
  sourceChain,
  factory.target,
  permit2Mock.target,
);
await router.waitForDeployment();
await factory.getFunction('vetInitialRouter')(router.target);

route = routed(router, {
  sourceChain,
  owner,
  portfolioContractAccount,
  AxelarGateway: axelarGateway,
  abiCoder,
});

multicallTarget = await (await ethers.getContractFactory('Multicall')).deploy();
```





```

    });

    it('executes the same instruction twice when delivered twice through the gateway', async
    () => {
        // Create the target RemoteAccount
        (await route(lca).doRemoteAccountExecute({ multiCalls: []
    })).expectOperationSuccess();

        const accountAddress = await route(lca).getRemoteAccountAddress();
        const mc = contractWithCallMetadata(
            makeEvmContract(multicallAbi,
                multicallTarget.target.toString() as `0x${string}`,
            );

        // Build a payload with a fixed txId and send it twice
        const txId = padTxId('duplicate-poc-0001', lca);
        const calls: ContractCall[] = [mc.addToValue(1n)];
        const payload = gmpRouterContract.processRemoteAccountExecuteInstruction(
            txId,
            accountAddress,
            { multiCalls: calls },
        );

        // Delivery #1
        const first = await route(lca).execRaw({ payload, txId });
        first.expectOperationSuccess();
        expect(await multicallTarget.getValue()).to.equal(1n);

        // Delivery #2
        const second = await route(lca).execRaw({ payload, txId });
        second.expectOperationSuccess();
        expect(await multicallTarget.getValue()).to.equal(2n);
    });
});

```

### RemoteAccountDuplicateSend.spec.ts

```

import AxelarGasService from '@axelar-network/axelar-cgp-solidity/artifacts/contracts/gas-
service/AxelarGasService.sol/AxelarGasService.json';
import { expect } from 'chai';

```



```
import { Contract } from 'ethers';
import { ethers } from 'hardhat';
import '@nomicfoundation/hardhat-chai-matchers';
import { HardhatEthersSigner } from '@nomicfoundation/hardhat-ethers/signers';
import { makeEvmContract } from '../utils/evm-facade';
import { contractWithCallMetadata } from '../utils/router';
import type { ContractCall } from '../interfaces/router';
import {
  deployRemoteAccountFactory,
  routed,
} from './lib/utils';
import { multicallAbi } from './interfaces/multicall';

describe('Unreachable SubcallOutOfGas PoC', () => {
  let owner: HardhatEthersSigner;
  let axelarGatewayMock: Contract;
  let factory: Contract, router: Contract, permit2Mock: Contract;
  let multicallTarget: Contract;

  const abiCoder = new ethers.AbiCoder();
  const sourceChain = 'agoric';
  const portfolioContractCaip2 = 'cosmos:agoric-3';
  const portfolioContractAccount = 'agoric1routerlca123456789abcdefghijklmnopqrstuvwxyz';
  const testLCA = 'agoric1oogbranch2test1234567890abcdefg';

  let route: ReturnType<typeof routed>;

  before(async () => {
    [owner] = await ethers.getSigners();

    const GasServiceFactory = await ethers.getContractFactory(
      AxelarGasService.abi, AxelarGasService.bytecode,
    );
    await GasServiceFactory.deploy(owner.address);

    const TokenDeployerFactory = await ethers.getContractFactory('TokenDeployer');
    const tokenDeployer = await TokenDeployerFactory.deploy();
```



```
const AuthFactory = await ethers.getContractFactory('AxelarAuthWeighted');
const authContract = await AuthFactory.deploy([
  abiCoder.encode(['address[]', 'uint256[]', 'uint256'], [[owner.address], [1],
1]),
]);

const AxelarGatewayFactory = await ethers.getContractFactory('AxelarGateway');
axelarGatewayMock = await AxelarGatewayFactory.deploy(
  authContract.target, tokenDeployer.target,
);

const MockPermit2Factory = await ethers.getContractFactory('MockPermit2');
permit2Mock = await MockPermit2Factory.deploy();

factory = await deployRemoteAccountFactory(
  portfolioContractCaip2, portfolioContractAccount, owner.address,
);

const RouterContract = await ethers.getContractFactory('RemoteAccountAxelarRouter');
router = await RouterContract.deploy(
  axelarGatewayMock.target, sourceChain, factory.target, permit2Mock.target,
);
await router.waitForDeployment();
await factory.getFunction('vetInitialRouter')(router.target);

route = routed(router, {
  sourceChain, owner, portfolioContractAccount,
  AxelarGateway: axelarGatewayMock, abiCoder,
});

const MulticallFactory = await ethers.getContractFactory('Multicall');
multicallTarget = await MulticallFactory.deploy();

const setup = await route(testLCA).doRemoteAccountExecute({ multiCalls: [] });
setup.expectOperationSuccess();
});

it('Branch 2 should revert with SubcallOutOfGas', async () => {
  const mc = contractWithCallMetadata(
```



```
        makeEvmContract(multicallAbi),
        multicallTarget.target.toString() as `0x${string}`,
    );

    // Tune gas to hit target branch
    const multiCalls: ContractCall[] = [
        mc.burnGas(50000n),
        mc.setValue.with({ gasLimit: 50n })(42n),
    ];

    const receipt = await route(testLCA, {
        async doExecute(commandId, sourceChain, sourceAddress, payload) {
            const gasEstimate = await this.execute.estimateGas(
                commandId, sourceChain, sourceAddress, payload,
            );
            return this.execute(commandId, sourceChain, sourceAddress, payload, {
                gasLimit: (gasEstimate * 70n) / 100n,
            });
        },
    }).doRemoteAccountExecute({ multiCalls });

    // Should revert with SubcallOutOfGas, but the decoding bug causes an empty revert
    and the assertion fails
    await expect(receipt).to.be.revertedWithCustomError(router, 'SubcallOutOfGas');
});
});
```

### SubcallOutOfGasUnreachable.spec.ts